(Chapter 6)

(Partitioning)

Breaking data into partitions is called sharding

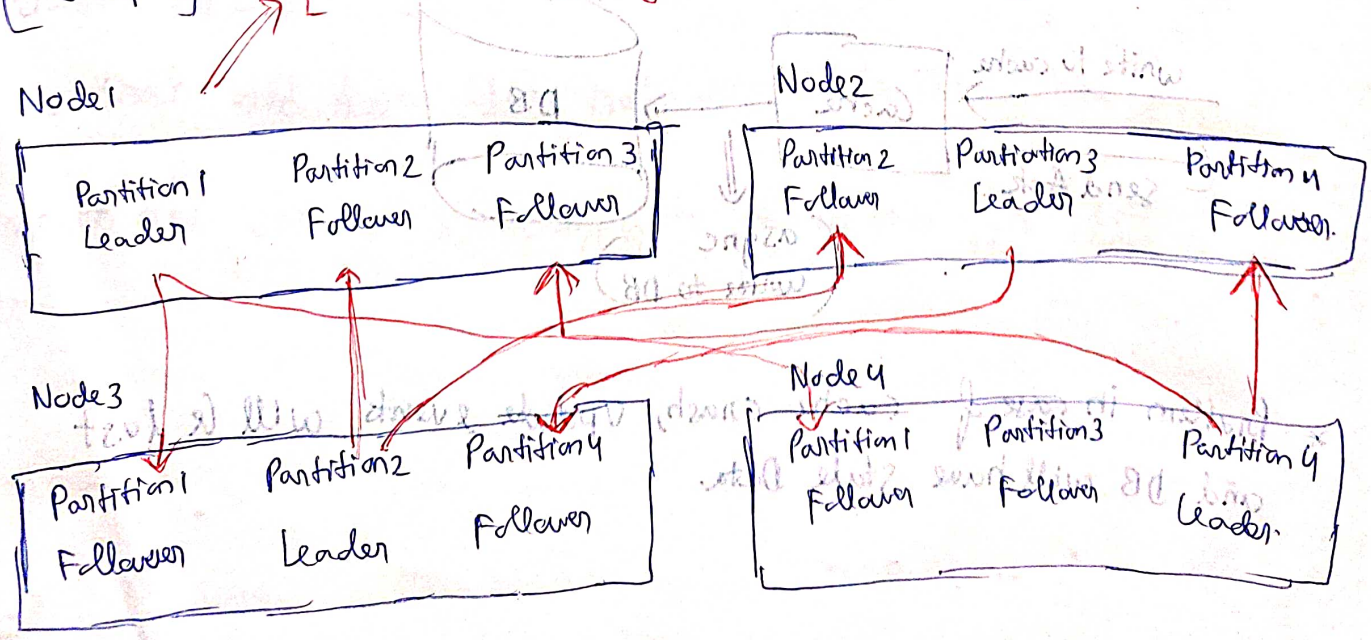
What we called a partition, is a shard in MongoDB and a vnode in Cassandra.

For queries that operate on a single partition, each node can independently execute the queries for its own partition, so query throughput can be scaled by adding more nodes.

Partition and Replication.

Database can be divided into partitions and each partition can be further ~~divided~~ replicated.

(Example) [Each node storing more than 1 partition]



Partitioning of Key-Value Data

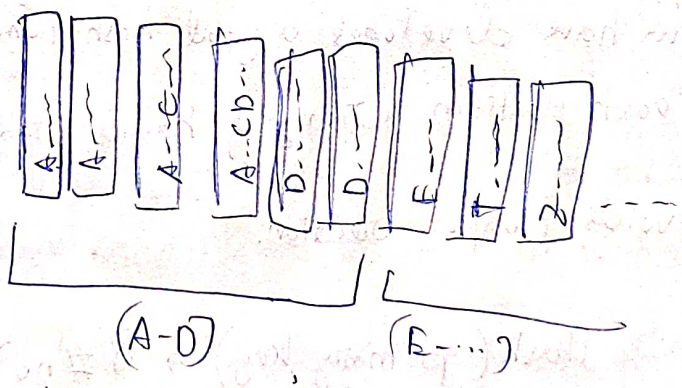
Our goal with partitioning is to spread the data and the query load evenly across nodes by partitioning.

If partitioning is unfair and in some cases, 1 node takes most of the load it's called skewed partition. A partition with disproportionately high load is called hotspot.

If we split data randomly we will never know which node to query, we will end up sending query to all nodes parallely

Partitioning by Key-Range

* In this case we can have partitions on the sorted Key Range.



* But the main part is deciding on the range.
Because we need to create the partitions and each partition should have the range in such a fashion such that there is no hotspot

* Within Each partition since data is sorted, We can keep multi-column indexes [concatenated index].
Based on highly queried columns.

(Disadvantage of Partition by Key Range)

If database's primary key is not designed correctly it will lead to timestamp.

If partition key is time stamp, then maybe write might happen only at a certain time of the day leading to hotspot.

Effective key creation is important, so that hotspot is not created

Partition by Hash of Key

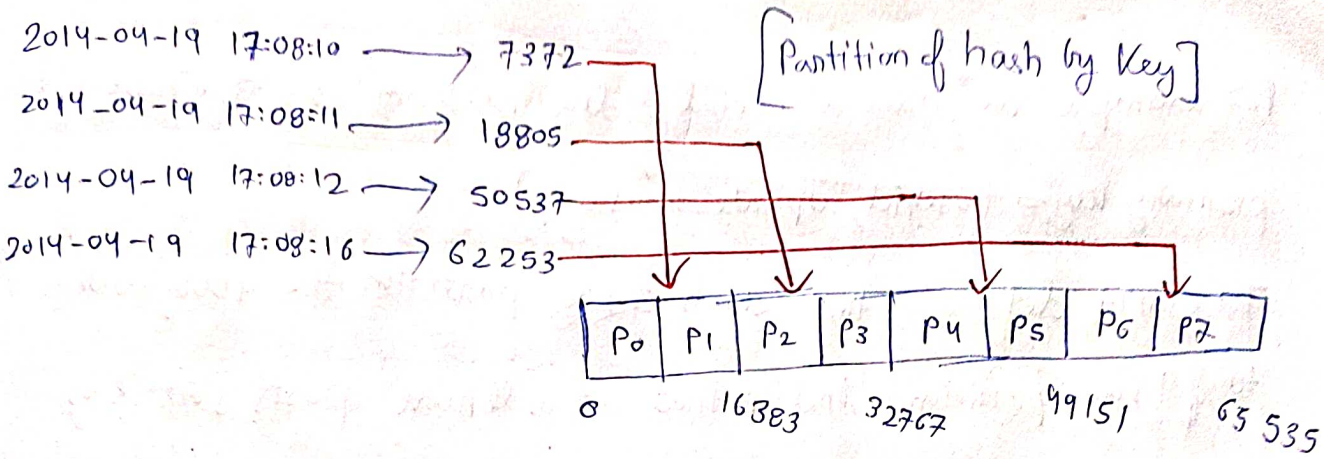
→ Avoiding Hotspot

Once we have developed a good hash-function for keys, we can assign each partition a range of hashes (rather range of keys).

This avoids Hotspot creation.

Hash (primary key) → (#no)

→ [Check which range it falls into]



[Consistent Hashing → Refer to Alex Xu book]

[Disadvantage of partitioning by range of HashKeys]

Unfortunately in this way, we loose the sorted order of keys and loose the capability of range queries. In case of range queries, we need to query all partitions.

(***) [How Cassandra deals with partition and Range Queries]

Cassandra achieves a compromise between two partitioning strategies

A table in Cassandra can be declared with a compound primary key, consisting of several columns. Only first part of ~~hashed key is used~~ key is hashed to determine the partition but other columns are used as concatenated index for sorting data in Cassandra SSTables.

So range query cannot be done on first column or partition key, but can be done on rest of the columns

[column1, column2, column3]
- column1 → Partition Key
- column2, column3 → Concatenated Index

Strategies for Rebalancing

* **(hash mod N)**

Range: $0 \leq \text{hash(key)} < b_0$ to partition1

$b_0 \leq \text{hash(key)} < b_1$ to partition2

Problem with modN is that no of nodes can change

In case of N changing, lot of keys will need migration to new Node, which we don't want to add.

* **(Fixed number of partitions)**

1) The partitions keep growing in size, as the dataset size is increasing. 1 node can have more than 1 partition.

1 node has 10 partitions each.

2) When a new node gets added some partition from each node get transferred to the new Node. Since the process takes time, the old assigned node is used for read and writes.

3) Partition size ∝ Size of cluster. Partition too big is tough to manage, partition too small, there is a large overhead.

Perfect size of partition is a bit hard to manage, if no of partitions is fixed and dataset size varies.

Dynamic Partitioning

* For DBs using key-range partioning, deciding partition size and partition (key-range) before hand can lead to hotspot creation and only 1 partition filling up. Reconfiguring the DB is tedious.

* For this reason partitions are dynamically managed split into two partitions and shrinked and merged with another partition if page becomes extra small.

  kind of B-tree, [page splitting / Page-merging]

[Caveat]: Initially DB starts with 1 node, 1 partition, As partition starts growing in size the partition has to split, which has a lot of overhead. So MongoDB in this case allows initial set of partitions on empty DB called pre-splitting

In Cassandra, to make number of partitions proportional to the number of nodes. In other words [no of partitions / no of nodes] = constant. To have fixed number of partitions per node. In this case, the size of the partition grows proportionally to the dataset size, while the number of nodes remain unchanged.

(P.T.O)

When a new node joins the cluster, it randomly chooses a fixed number of partitions to split, then takes ownership of one half of each of those split partitions, while leaving the other half in place

This algo of splitting partitions randomly can lead to unfair splits; ~~but when averaged over a large~~

This algo of splitting partitions randomly is replaced by Consistent Hashing [Including Virtual Nodes] in Cassandra 3.0, which makes splitting fair.

Automatic or Manual Rebalancing

In case of automatic rebalancing system decides automatically when to move partitions from one node to another.

Manual Rebalancing → Admin has control

Fully automatic is fine and but dangerous. Suppose if one node is overloaded and is temporarily slow to respond to requests. the other nodes conclude that node is dead and they try to rebalance and this puts additional load on the system.

So its better to have some manual intervention.

Request Routing.

Q) When a client wants to make a request, How does it know which node to connect to?

If I want to get a node in which I can find the key = "foo". which IP address do I use?

This problem is critically solved by service discovery.
Many companies have their own in-house service discovery tools.

(On high level) (Few options)

1) Allow clients to ~~conn~~ connect to any node, and the node which has the actual data, its data we can get from any node and its traffic is directed to that node.
Basically each node has info about mapping

2). Routing tier / Load Balancer which has all the info and directly directs to actual traffic

3). User has data pre defined and directly provides IP address of the server where data is residing.

In all of the approaches mentioned above we still want to know after rebalancing, how is the re-routing happening

Many distributed data systems use Zookeeper to maintain this detail and keep track. Each node, old or newly created registers itself into the Zookeeper, time to time

Cassandra on the other hand does not use Zookeeper It uses gossip protocol such that each node knows mappings of one another. Requests are sent to any node and that node redirects traffic to the node which has the data.

Parallel Query Execution.

MPP → Massively Parallel Processing relational database Products (Chapter 10 for Parallel Processing).

Partitioning And Secondary Indexes

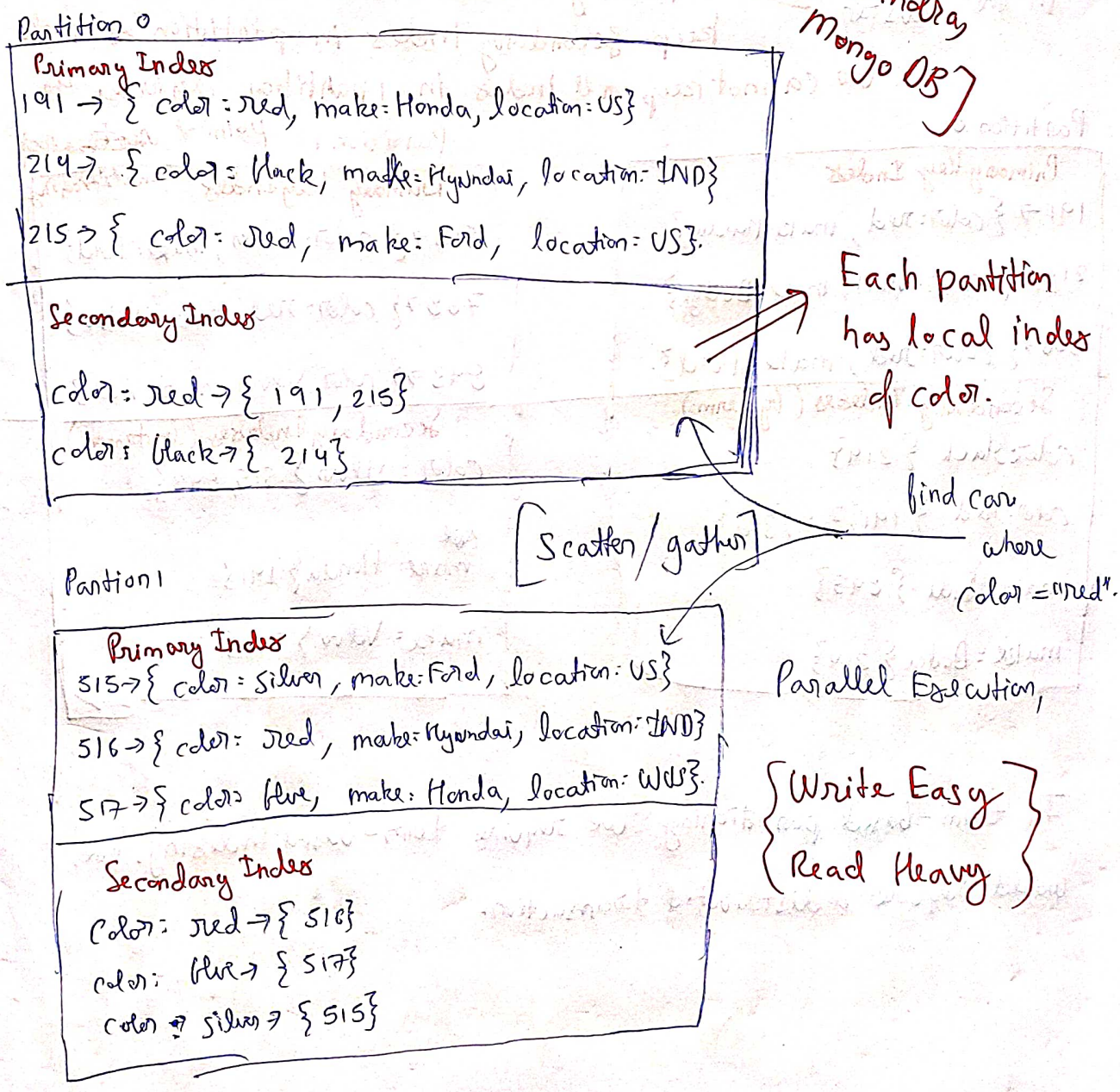
Lets say we created partitions based on primary key only. But now the query comes on a column which is not primary key and most queries are made by the user on that column. Then we need to create Secondary Indexes on each partition, So our search becomes easy.

The problem with Secondary Indexes is that it does not map directly to partitions. Unlike primary keys which determine partition number, secondary index cannot create a mapping to partition no.

Index is distributed across nodes in cluster, so every node will be queried.

Partitioning Secondary Index by Document

=> Used in Cassandra, Mongo DB



Partitioning Secondary Index by Term.

↳ {Write Heavy, Read Faster}

This is something really innovative and amazing at the same time.

In this case we don't have a local secondary index restricted to each partition, rather we have a global secondary index, which indexes the data of all the partitions in the cluster. (***) Imp thing to note is that we will have to keep secondary index in partition as well we cannot keep all index in 1 partition, defeats the point of partitioning.

Partition 0

Primary Key Index

191 → { color: red, make: Honda }

214 → { color: black, make: Dodge }

306 → { color: red, make: Ford }.

Secondary Indexes (by term)

color: black { 214 }

color = red { 191, 768, 306 }

make = Audi { 893 }

make = Dodge { 214 }

Partition 1

Primary Key Index

515 → { color = Silver, make: Ford }

768 → { color = red, make = Volvo }

893 → { color: Silver, make = Audi }

Secondary Indexes (by term)

color = Silver { 515, 893 }

make: Honda { 191 }

make = Volvo { 768 }

In term-based partitioning we require term-based indexing. We would require a distributed transaction.